

# MIP\_WORKSPACE\_BOOTSTRAP.md

## Mainframe Intelligence Platform

### Workspace Bootstrap Guide

Version: 1.0

---

## Purpose

This document defines the standard engineering workspace for all MIP contributors.

The objective is to ensure every engineer, architect, analyst, and AI-assisted workflow operates consistently across the platform.

This guide must be followed before writing any code.

---

## Guiding Principle

MIP is not an AI project.

MIP is an Enterprise Knowledge Extraction Platform.

The implementation sequence is:

Source Code ↓ Metadata ↓ Relationships ↓ Knowledge Graph ↓ Reasoning ↓ Modernization  
Intelligence

Every engineering decision should reinforce this sequence.

---

## Prerequisites

### Software

Required:

- Git
- Python 3.13+
- VS Code
- GitHub Copilot
- SQLite Browser (optional)

Recommended:

- Draw.io
  - DBeaver
  - Docker Desktop
- 

## VS Code Extensions

Mandatory:

- GitHub Copilot
- GitHub Copilot Chat
- Python
- Pylance
- Ruff
- Markdown All In One

Recommended:

- Draw.io Integration
  - SQLite Viewer
  - YAML
  - JSON Tools
- 

## Repository Layout

```
mip-platform/  
  
.github/  
  
skills/  
  
prompts/  
  
docs/  
  
src/  
  
tests/  
  
output/  
  
examples/  
  
scripts/
```

```
tools/
```

```
data/
```

```
logs/
```

---

## Python Environment

Create:

```
python -m venv .venv
```

Activate environment.

Install:

```
pip install fastapi  
pip install pydantic  
pip install networkx  
pip install pytest  
pip install uvicorn  
pip install sqlalchemy  
pip install ruff
```

---

## Workspace Directories

### **docs/**

All architecture and design artifacts.

Examples:

- Architecture
- Roadmaps
- Standards
- Decisions

---

### **skills/**

Reusable Copilot skills.

Examples:

- Repository Engineer
  - Mainframe Architect
  - Code Analyst
- 

## **prompts/**

Reusable prompt library.

Organized by domain.

---

## **src/**

Production source code.

Contains:

- discovery
  - parsers
  - metadata
  - graph
  - api
  - services
- 

## **tests/**

All automated tests.

No production code.

---

## **output/**

Generated artifacts.

Examples:

- Inventory
  - Metadata
  - Graph exports
-

## data/

Local SQLite databases.

Examples:

```
mip.db  
metadata.db
```

---

## logs/

Application logs.

---

# Copilot Setup

Create:

```
.github/
```

Add:

```
copilot-instructions.md
```

This becomes the global engineering contract.

---

# Skill Installation

Required Skills:

- repository-engineer
  - mainframe-modernization-architect
  - mainframe-code-analyst
  - metadata-modeler
  - graph-engineer
  - sqlite-engineer
  - test-engineer
  - code-reviewer
  - documentation-writer
-

# Prompt Installation

Required Libraries:

- architecture
  - discovery
  - parsing
  - metadata
  - graph
  - testing
  - review
  - modernization
- 

## Development Workflow

Every feature follows:

Design ↓ Review ↓ Critique ↓ Improve ↓ Implement ↓ Test ↓ Refactor ↓ Document

Never skip design.

Never skip testing.

---

## Quality Gates

Before merge:

- Architecture reviewed
  - Tests pass
  - Ruff passes
  - Documentation updated
  - Metadata model reviewed
- 

## First Run Checklist

Create repository.

Create workspace.

Install extensions.

Install dependencies.

Install skills.

Install prompt library.

Create SQLite database.

Commit baseline.

Tag:

v0.1-bootstrap

---

## Definition of Ready

A feature is ready when:

- Requirements exist
  - Metadata model exists
  - Acceptance criteria exist
  - Test strategy exists
- 

## Definition of Done

A feature is done when:

- Code implemented
  - Tests passing
  - Documentation updated
  - Metadata generated
  - Review completed
- 

## Success Metric

A new engineer can clone the repository and become productive within one day.